**Code Jam 2015 - Round 1C**

# Analysis: Brattleship

Once we first make a hit, it will take at least W-1 more moves to win, since we have to hit the remainder of the ship.

If there's still more than one possible position for the ship, then the little brother will get at least one more opportunity to answer "miss." To limit the number of additional misses to one, we make moves adjacent to the hits we already have. If we get a "miss," then we will have a row of "hits" with a miss at the end, so we will know the exact location of the ship.

Now, the little brother may as well try to maximize the number of misses we make until we first get a hit. He can't control what cells we name, so all he can do is answer "miss" until we make a guess which must unavoidably be a hit. So we need to find a pattern of guesses that uses as few cells as possible, and such that each possible ship position is covered by one of them. To do this, we use a pattern that chooses every $W^{th}$ cell of each row.

The total number of guesses is then R * floor(C/W) for the pattern, plus W-1 to hit the remainder of the ship, plus 1 more guess if there is more than one possibility for the position of the ship, which occurs if C is not an exact multiple of W. Below is a sample code in C that implements this solution:

```
#include <stdio.h>

int main() {
  int T, TC, R, C, W, score;

  scanf("%d", &T);
  for (TC = 1; TC <= T; TC++) {
    scanf("%d %d %d", &R, &C, &W);

    // The R * floor(C/W) for the guess pattern.
    score = R * (C / W);

    // Plus W-1 to hit the remainder of the ship.
    score += W - 1;

    // Plus 1 more guess if there is more than one
    // possibility for the position of the ship,
    // which occurs if C is not an exact multiple of W.
    if (C % W) score++;

    printf("Case #%d: %d\n", TC, score);
  }
}
```

tos.lunar wrote a solution for this question using Haskell, which you can download from the scoreboard.

A recursive search of the game tree, simulating all choices for our moves and the little brother's moves, will also work for the small input.

**Code Jam 2015 - Round 1C**

# Analysis: Less Money, More Problems

We will incrementally build a set S of denominations that solves the problem using the minimal number of additional denominations, by restricting ourselves to adding denominations from smallest to largest.

As we add denominations to S, we also maintain an integer N, which is the largest value such that we can produce each value up to and including N. In fact, after all of our choices, S will be able to produce exactly the set of values from 0 to N, and no others.

When we add a new denomination X to S, the new set of values we could produce include each of the values we could produce with the existing set S plus between 0 and C of the new denomination X. If X is at most N+1, then this new set of values will be the set of all values from 0 to N+X*C, so we can update N to N+X*C.

So, we initialize S to the empty set, and N to 0.

Then while N is less than V, we do the following:

- Identify the smallest value we cannot produce: N+1.
- If there is still a pre-existing denomination which we haven't used, let the minimum such denomination be X. If X is less than or equal to N+1, we add it to S, and update N to N+X*C.
- Otherwise, we have no way yet to produce N+1 using the denominations we have, so we must add to S a new denomination X between 1 and N+1. This will increase N to N+X*C. We use X=N+1. No other choice for X could lead to a better solution, since for X=N+1, the set of values the new S will be able to produce is a superset of the values S would be able to produce with any other choice.

Finally, when we have a set S which can produce all values up to V, we output the number of new denominations we had to add.

In the above algorithm, the first option — using a pre-existing denomination — can only occur D times. When the second option is chosen, N increases to (C+1)N+C. Since we stop when N reaches V, this will occur O(log V) times. So the overall time complexity is O(D+log(V)).

Sample implementation in Java:

```
import java.util.*;

public class C {
  public static void main(String[] args) {
    Scanner scan = new Scanner(System.in);
    int T = scan.nextInt();
    for (int TC = 1; TC <= T; TC++) {
      int C = scan.nextInt();
      int D = scan.nextInt();
      int V = scan.nextInt();
      Queue<Integer> Q = new ArrayDeque<>();
      for (int i = 0; i < D; i++) {
        Q.add(scan.nextInt());
      }
```

```
      long N = 0;
      int add = 0;
      while (N < V) {
        // X = The smallest value we cannot produce.
        long X = N + 1;
        if (!Q.isEmpty() && Q.peek() <= X) {
          // Use pre-existing denomination we haven't used.
          X = Q.poll();
        } else {
          // No way to produce N+1, add a new denomination.
          add++;
        }
        N += X * C;
      }
      System.out.printf("Case #%d: %d\n", TC, add);
    }
  }
}
```

Vitaliy's solution in C, which you can download from the scoreboard, is another good example of this approach.

**Code Jam 2015 - Round 1C**

# Analysis: Typewriter Monkey

This problem naturally has two parts — computing the maximum number of bananas needed, and computing the expected number of bananas needed.

Take, for example, the target string X = ABACABA. To find a string of length S containing the maximum number of copies of X, we start by putting X at the start of the string. Then to fit as many more copies as possible, we want to overlap each copy of X as much as possible with the previous copy. For this example, we could overlap with the final "A" and add "BACABA", but it is even better to overlap with "ABA" and add just "CABA" to get a second copy. To find the maximum amount of overlap, we can just try every possible amount and check which ones work, since L is small. It can also be computed in linear time using the initialization phase of the Knuth-Morris-Pratt algorithm. If the maximum amount of overlap is O, then we can fit 1+(S-L)/(L-O) copies of the string.

To find the expected number of copies, we start by computing the probability P of the word occurring at a fixed place. This is equal to the product of the probabilities for each letter of the word being correct. The probability for a single letter being correct is the fraction of keys which are that letter.

By linearity of expectation, the expected number of copies is then just P multiplied by the number of places the string can occur, which is S-L+1. This is a convenient fact to use, because we don't need to take into account that the string occurring in one position and the string occurring in an overlapping position are not independent events.

Sample implementation in Python:

```
# Find the maximum amount of overlap. We can just try
# every possible amount and check which ones work.
def max_overlap(t):
  for i in range(1, len(t)):
    if t[i:] == t[0:len(t)-i]:
      return len(t) - i
  return 0

# Returns the probability of the target word
# occurring at a fixed place.
def probability(target, keyboard):
  P = 1.0
  # Compute the product of the probabilities
  # for each letter of the word being correct.
  for i in range(len(target)):
    # The probability for a single letter being correct
    # is the fraction of keys which are that letter.
    C = keyboard.count(target[i])
    P = P * C / len(keyboard);
  return P

for tc in range(input()):
  K, L, S = map(int, raw_input().split(' '))
  keyboard = raw_input()
  target = raw_input()
  res = 0
```

```
  P = probability(target, keyboard)
  if P > 0:
    O = max_overlap(target)
    max_copies = 1.0 + (S-L) / (L-O)
    min_copies = P * (S-L+1)
    res = max_copies - min_copies
  print("Case #%d: %f" % (tc + 1, res))
```

Klockan wrote a solution in C++ that also uses this approach, which you can download from the scoreboard.

We could also use a dynamic programming algorithm for both parts of the problem, using O(LS) states, where the state is the number of characters typed and the largest number of characters of the word that are currently matched, and the value at each state is the probability of that state and the maximum number of copies of the word that could have been produced while reaching it. For each of the states where the entire word has just been matched, we add the probability of reaching that state to the expected number of copies of the word, and update the maximum number of copies that are possible. linguo wrote a solution of this type in Python.